In other words, TypeProf tracks type information through the code, using variable assignments and what it knows about method calls.

For example, if you run the following code through TypeProf:

```
def approximate_word_count(sentence)
  sentence.split(/\W+/).size
end

approximate_word_count("This is a sample word count")
```

TypeProf will infer that sentence is a string from the literal assignment, and then it will walk through the known type signatures in the method. The split method takes a string and returns an array of strings, the size method takes an array and returns an integer, therefore TypeProf deduces that approximate_word_count takes a string argument and returns an integer.

TypeProf has some limitations. If there's no call to a method in the code, or no assignment in the code, then it's limited in how much information it has and will only provide very limited and probably overly general results. Metaprogramming will often confuse TypeProf, especially if a lot of the data is unknown at load time (so it might be able to manage a define_method over a known array, but send where the argument is a variable will confound it).

TypeProf continues to be under active development, an up-to-date description of changes can be found at https://github.com/ruby/typeprof/blob/master/doc/doc.md.

Advanced RBS Syntax

RBS syntax, for the most part, uses : CLASS after method names, argument names, and attribute name, and the -> CLASS syntax for the return values of methods. There are some cases where that's not enough to capture the complexity of a Ruby program, so there's some additional syntax to be aware of. RBS syntax may yet change, and there are some complexities we don't get to here, so be sure to check out the final source of documentation at https://github.com/ruby/rbs/blob/master/docs/syntax.md.

In general, the type of an object is its class name, but there are some special cases. You can reflexively refer to the type of the receiver inside a class with self and to the singleton object of the class with singleton. The class itself is accessible with class and instances of the class are accessible as instance.

Sometimes a proc or method is used as a return value or argument. The syntax is ^(ARGLIST) -> RETURN_TYPE, as in:

```
def apply_a_function: (^(String, Integer) -> String) -> String
```

In this case, the argument to apply_a_function is a function that takes a string and an integer argument and returns a string.

You can use alias to give a new name to an existing name or pattern, so you could simplify the above as:

```
alias func ^(String, Integer) -> String
def apply_a_function: (func) -> String
```

In some cases, you might have a variable that is more than one type. For example, we sometimes write methods that take either an ID pointing to an object in a database or an